Leveraging Machine Learning for Gate-level Timing Estimation Using Current Source Models and Effective Capacitance

Dimitrios Garyfallou
Dept. of Electrical & Computer Eng.
University of Thessaly, Volos, Greece
digaryfa@e-ce.uth.gr

Anastasis Vagenas
Dept. of Electrical & Computer Eng.
University of Thessaly, Volos, Greece
avagenas@e-ce.uth.gr

Charalampos Antoniadis
CEMSE Division
KAUST, Thuwal, Saudi Arabia
charalampos.antoniadis@kaust.edu.sa

Yehia Massoud
CEMSE Division
KAUST, Thuwal, Saudi Arabia
yehia.massoud@kaust.edu.sa

George Stamoulis
Dept. of Electrical & Computer Eng.
University of Thessaly, Volos, Greece
georges@e-ce.uth.gr

ABSTRACT

With process technology scaling, accurate gate-level timing analysis becomes even more challenging. Highly resistive on-chip interconnects have an ever-increasing impact on timing, signals no longer resemble smooth saturated ramps, while gate-interconnect interdependencies are stronger. Moreover, efficiency is a serious concern since repeatedly invoking a signoff tool during incremental optimization of modern VLSI circuits has become a major bottleneck. In this paper, we introduce a novel machine learning approach for timing estimation of gate-level stages using current source models and the concept of multiple slew and effective capacitance values. First, we exploit a fast iterative algorithm for initial stage timing estimation and feature extraction, and then we employ four artificial neural networks to correlate the initial delay and slew estimates for both the driver and interconnect with golden SPICE results. Contrary to prior works, our method uses fewer and more accurate features to represent the stage, leading to more efficient models. Experimental evaluation on driver-interconnect stages implemented in 7 nm FinFET technology indicates that our method leads to 0.99% (0.90 ps) and 2.54% (2.59 ps) mean error against SPICE for stage delay and slew, respectively. Furthermore, it has a small memory footprint (1.27 MB) and performs 35× faster than a commercial signoff tool. Thus, it may be integrated into timing-driven optimization steps to provide signoff accuracy and expedite timing closure.

CCS CONCEPTS

• **Hardware → Electronic design automation**; **Timing analysis**;
• **Computing methodologies → Machine learning**.

KEYWORDS

Timing analysis, machine learning, current source models, effective capacitance, resistive shielding, Miller effect

GLSVLSI '22, June 6–8, 2022, Irvine, CA, USA

ACM ISBN 978-1-4503-9322-5/22/06…$15.00
https://doi.org/10.1145/3526241.3530343

ACM Reference Format:
Dimitrios Garyfallou, Anastasis Vagenas, Charalampos Antoniadis, Yehia Massoud, and George Stamoulis. 2022. Leveraging Machine Learning for Gate-level Timing Estimation Using Current Source Models and Effective Capacitance. In *Proceedings of the Great Lakes Symposium on VLSI 2022 (GLSVLSI '22), June 6–8, 2022, Irvine, CA, USA.* ACM, New York, NY, USA, 7 pages. https://doi.org/10.1145/3526241.3530343

1 INTRODUCTION

Timing analysis is an essential and demanding verification method used in the initial design and iterative optimization of a Very Large Scale Integrated (VLSI) circuit, while it also constitutes the cornerstone of the final signoff. In practice, timing analysis is performed at the gate level, where circuit delay is analyzed in stages [1]. Each stage consists of a driver gate, a distributed *RC* interconnect, and multiple receiver gates. However, in advanced technology nodes, accurate and efficient analysis of gate-level stages becomes even more challenging, since VLSI interconnects are increasingly resistive, signals no longer resemble smooth saturated ramps, and gate inputs exhibit significant Miller effect [2]. Moreover, gate-interconnect interdependencies render an iterative approach mandatory.

Over recent years, Current Source Models (CSMs) have been proposed for accurate gate modeling, which capture more detail compared to the conventional Non Linear Delay Model (NLDM). However, industrial CSMs [3, 4] are precharacterized into standard cell libraries assuming lumped capacitive loads, which poses significant challenges to the approximation of the driving point admittance of highly resistive interconnects with an effective capacitance (C_{eff}) [5]. Traditional C_{eff} estimation techniques [5, 6] focus on computing a single capacitance value for accurate gate delay estimation, thus being inadequate to approximate slew. To accurately estimate the nonlinear driver output waveform using CSMs, C_{eff} must be computed in multiple voltage regions [7].

On the other hand, timing analysis of contemporary on-chip interconnects has become crucial, as interconnect delay represents an increasingly dominant portion of stage delay [8]. SPICE simulation provides golden results but is prohibitive for iterative optimization flows, while fast moment-based timing metrics [9–12] may be quite inaccurate, especially for *RC* structures with many branches routed on different metal layers, as they rely on simplistic assumptions.

To address the challenges existing in gate-level timing analysis, a signoff Static Timing Analysis (STA) tool is employed throughout

the design flow. Although it is more efficient than SPICE, the interaction with a signoff tool during iterative optimization of modern VLSI circuits has become the bottleneck of physical implementation. To sidestep this limitation, recent research endeavors [13–15] resort to Machine Learning (ML) techniques. In [13], ML models based on Least-Squares Regression (LSQR) are proposed to predict interconnect timing, using several delay and slew metrics (e.g., [9, 10]) as features. Authors in [14] extract topological features to capture the characteristics of *RC* interconnects and exploit XGBoost [16] for fast timing prediction. Common to the above methods [13, 14] is that they focus only on interconnect analysis, assuming that accurate gate timing estimation can be straightforwardly obtained by using the precharacterized gate models. However, as mentioned above, accurate C_{eff} modeling is not a trivial task. In a different approach, authors in [15] attempt to predict timing at the pre-routing step, neglecting the *RC* parasitics introduced during Place and Route (PnR).

In this paper, we introduce a novel ML-enhanced approach for timing estimation of gate-level stages, which achieves signoff accuracy and is highly efficient for optimization flows. Our main idea is to exploit a fast iterative algorithm that provides initial delay and slew estimates for the driver and the interconnect while extracting a small number of features representing the stage, and then employ ML models to correlate these estimates with SPICE. The contributions of this work can be summarized as follows:

- We develop an iterative algorithm that approximates the nonlinear voltage waveforms by piecewise linear (PWL) ramps, using CSMs and multiple C_{eff} values while considering the Miller effect. Compared to prior works that compute C_{eff} using a π-model approximation of the interconnect [7, 17], our algorithm provides better accuracy and efficiency, as it avoids such an approximation.
- We propose a new ML feature extraction approach that effectively captures the characteristics of gate-level stages. This is the first study that explores the concept of multiple slew and C_{eff} values to build accurate ML models for post-PnR timing prediction. Compared to the state-of-the-art approach for interconnect analysis [14], our method uses fewer and more accurate features to represent the stage, thus leading to more efficient models.
- We present four ML models to predict delay and slew for both the driver and interconnect of a stage. To this end, we evaluate different ML engines, including linear regression (LASSO), XGBoost, and Artificial Neural Networks (ANNs). Unlike [13–15] that aim to correlate with a signoff STA tool (ST), we train our models to predict golden SPICE values, while we also adjust them to incline toward a pessimistic prediction for worst-case timing analysis. The proposed models can be trained once and applied to different designs using the same manufacturing process technology.
- We demonstrate the efficacy of our ML-assisted approach on representative stages implemented in 7 nm FinFET technology [18], where accurate timing and C_{eff} estimation is increasingly challenging. Results indicate that our method exhibits a strong correlation with SPICE (0.999) and similar accuracy with ST, achieving 0.99% (0.90 ps) and 2.54% (2.59 ps) mean error against SPICE for stage delay and slew, respectively. Moreover, it has a small memory footprint (1.27 MB) while performing 35× faster than ST.

The rest of the paper is organized as follows. In Section 2, we define the problem formulation and provide background on stage timing analysis along with our extensions for multiple C_{eff} and slew computation. Section 3 presents our ML-enhanced approach for timing estimation of gate-level stages using CSMs and C_{eff}. In Section 4, we compare against other timing estimation methods and report results for representative stages. Finally, Section 5 concludes this work.

2 TIMING OF GATE-LEVEL STAGES

Gate-level timing analysis (i.e., STA [1] or DTA [19]) of VLSI circuits is performed in stages. First, the interconnect input admittance is approximated by C_{eff} and the driver output voltage waveform is computed in order to estimate driver delay and output slew. Then, this waveform (or the estimated slew value(s)) is used for interconnect delay estimation as well as for estimation of the receiver input voltage waveform (or slew value(s)). The receiver input slew is then used as input in the analysis of the next stage.

2.1 Problem formulation

Consider the stage of Figure 2, where the driver gate (i.e., the source node 0) is connected to the receiver gates with a distributed interconnect. Each node i of the RC-tree has a capacitance to ground C_i and is connected with a resistance $R_{i\rightarrow j}$ to each fanout node j. We denote the slew at node i by S_i, its total downstream capacitance (i.e., the sum of all the capacitances of the subtree starting at node i) by C_{tot_i}, and the corresponding effective capacitance by C_{eff_i}.

Problem: The objective of this work is to accurately estimate the driver delay (D_{drv}), the driver output slew (S_{drv}), the interconnect delay to the target receiver (D_{rcv}), and the target receiver input slew (S_{rcv}), given the driver input slew ($S^{\text{in}}_{\text{drv}}$) and the output capacitance values ($C^{\text{out}}_{\text{rcv}}$) at the receivers [typically set to the total capacitance (C_{total}) of the respective fanout stage]. [1]

2.2 Gate timing

2.2.1 ***Library compatible CSMs.*** Although several CSMs have been proposed in the literature, our methodology focuses on *library compatible CSMs*, such as the Effective Current Source Model (ECSM) [3] and the Composite Current Source (CCS) model [4], which are precharacterized into Look-Up Tables (LUTs) of technology libraries to be used for stage delay and slew estimation. Below, we provide a brief demonstration of the CCS model.

CCS driver model captures the output current as a function of driver input slew ($S^{\text{in}}_{\text{drv}}$) and output load ($C^{\text{out}}_{\text{drv}}$). As shown in Figure 1, this information is stored in `output_current_rise/fall` LUTs. The time instant when the corresponding driver input voltage crosses the delay threshold (usually $0.5V_{\text{dd}}$), which is necessary to calculate gate delay, is also stored as `reference_time` in these LUTs.

CCS receiver model typically uses two different input pin capacitance (C_{p}) values, $C1$ and $C2$, to model the nonlinear receiver transistor input capacitance and the Miller effect. This is very important since the load seen by the driver depends both on the interconnect *RC* network and the input pin capacitance of the multiple receiver gates. As depicted in Figure 1, CCS receiver capacitance values are stored in `receiver_capacitance_1/2_rise/fall` LUTs, and are also dependent on the input slew ($S^{\text{in}}_{\text{rcv}}$) and the output load ($C^{\text{out}}_{\text{rcv}}$) of the receiver cell.

[1] The problem addressed in this work was also the topic of the ACM TAU 2021 contest [20].

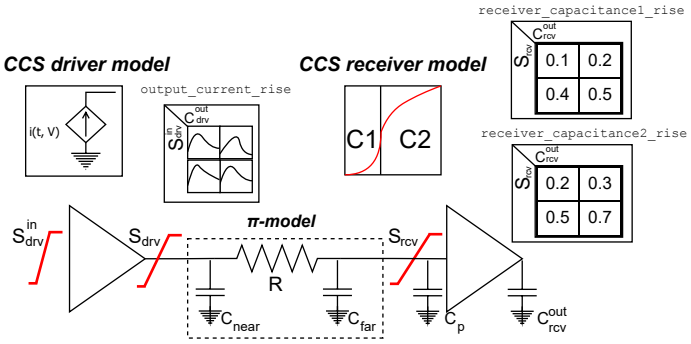

Figure 1: The CCS timing model and a reduced π-model load.

2.2.2 ***Modeling the* RC *interconnect load with* C_{eff}.** In advanced technology nodes, on-chip *RC* interconnect loads become increasingly resistive, which complicates their modeling with a single capacitance value for gate timing estimation with CSMs. The simplest approximation of the driving point admittance of an *RC* interconnect is C_{total}. However, this results in pessimistic gate timing estimation, as it totally ignores the resistive shielding effect, where interconnect resistance shields a part of C_{total}.

To address this limitation, the concept of C_{eff} is introduced in [5], which is an equivalent capacitance such that gate timing is the same when using (i) the driving point admittance and (ii) the estimated C_{eff} as a load. The traditional approach for C_{eff} estimation includes two steps [5]. First, the distributed interconnect is approximated by a π-model, proposed by O'Brien and Savarino [17], such as the π-model depicted in Figure 1. Then, C_{eff} is computed by equating the charge absorbed by the π-model and the charge absorbed by C_{eff}, up to the delay threshold (i.e., the time instant when driver output voltage crosses $0.5V_{\text{dd}}$). However, a single C_{eff} value focuses only on delay estimation and is inadequate to approximate slew, as it assumes that driver output voltage waveform is a linear ramp. Moreover, it does not consider the Miller effect, neglecting the impact of receiver input capacitance (C_{p}) on C_{eff}, delay, and slew estimation.

In order to accurately approximate the nonlinear driver waveform, C_{eff} must be computed in multiple voltage regions. Recently, authors in [7] presented a CSM-based method to compute driver output C_{eff} and slew in multiple regions, considering the Miller effect, for a π-model load approximation. The main drawback of [7] is that the reduction of the distributed interconnect to a π-model induces significant error in advanced technologies while introducing runtime and memory overhead, as we demonstrate in Section 4.

Another interesting C_{eff} estimation technique [6] handles the distributed interconnect as connected π-models and iteratively refines C_{eff} (along with delay and slew) on each RC-tree node by repeated forward-backward traversals. However, it does not exploit CSMs and still approximates signal waveforms by linear ramps, assuming a single C_{eff} on each node while ignoring the Miller effect.

To support our methodology, we extended [6, 7] to compute multiple C_{eff} values, avoiding a π-model reduction. In our modeling, for each node i of the *RC*-tree (see Figure 2), C_{eff_i} for each region $(\alpha V_{\text{dd}} - \beta V_{\text{dd}})$, where $\alpha, \beta \in [0.0, 1.0]$ and $\alpha < \beta$, is calculated by:

$$C_{\text{eff}_i}^{\alpha-\beta} = C_i + \sum_{\text{each fanout } j} K_j^{\alpha-\beta} * C_{\text{tot}_j}, \tag{1}$$

where $K_j^{\alpha-\beta} = 1 - \frac{X_j^{\alpha-\beta}}{(\beta-\alpha)}\left(e^{-\frac{\alpha}{X_j^{\alpha-\beta}}} - e^{-\frac{\beta}{X_j^{\alpha-\beta}}}\right)$ and $X_j^{\alpha-\beta} = \frac{R_{i\to j} C_{\text{eff}_j}^{\alpha-\beta}}{S_i^{\alpha-\beta}}$.

In the above formula, $S_i^{\alpha-\beta}$ is the slew of node i while $C_{\text{eff}_j}^{\alpha-\beta}$ is the C_{eff} of the fanout node j, corresponding to region $(\alpha V_{\text{dd}} - \beta V_{\text{dd}})$. As can be seen, $C_{\text{eff}_i}^{\alpha-\beta}$ accounts for a different shielding factor ($K_j^{\alpha-\beta}$) for each fanout node j, which shields C_{tot_j} and depends on $R_{i\to j}$, $C_{\text{eff}_j}^{\alpha-\beta}$, and $S_i^{\alpha-\beta}$ to properly account for the resistive shielding. Note that only for receiver input pins, C_i also incorporates the receiver input capacitance (including the Miller component) computed in the corresponding region ($C_p^{\alpha-\beta}$).

The main advantage of using multiple C_{eff} regions for each *RC*-tree node is that the nonlinear driver output waveform can be accurately approximated by a PWL ramp, exploiting library compatible CSMs to compute the slew of each region using the corresponding C_{eff} value. As a result, our approach achieves superior accuracy compared to [6, 7], especially for driver output slew, while it is also essential for accurate interconnect analysis.

2.3 Interconnect timing

The most accurate interconnect timing estimation is obtained by transient analysis using SPICE. Although SPICE provides golden results, it fails to meet the performance and memory requirements for full-chip analysis since it involves detailed computations with runtime proportional to the number of time steps and the *RC* network complexity. On the contrary, several lighter analytical models, based on the first few moments of the impulse response, have been proposed for fast timing analysis of interconnects [9–12]. However, these models assume step inputs while ignoring the resistive shielding and the interdependence between slew and C_{eff}.

In contrast to conventional delay and slew models [9–12], the iterative refinement-based method proposed in [6] exploits the C_{eff} concept for accurate gate-level interconnect timing estimation. More specifically, interconnect delay is computed by an extended version of Elmore delay, which uses C_{eff_i} instead of C_{tot_i} on each interconnect node i. For the *RC*-tree of Figure 2, the delay from driver output (node 0) to each node i is computed by:

$$D_{0\to i} = \sum_{\text{each node } k} R_{ki} C_{\text{eff}_i}^{0-0.5}, \tag{2}$$

where R_{ki} is the common resistance of paths $0 \to i$ and $0 \to k$. On the other hand, in [6], a single slew value is computed on each node, using $C_{\text{eff}}^{0-0.5}$ and ignoring the Miller capacitance, which is insufficient to accurately approximate the nonlinear signal waveform.

To enable accurate stage timing analysis, we generalized the slew metric of [6] to any voltage region, considering the respective C_{eff} [computed by Eq. (1)]. In our method, for each node i, the slew of region $(\alpha V_{\text{dd}} - \beta V_{\text{dd}})$ is propagated to each fanout node j as follows:

$$S_{\text{j}}^{\alpha-\beta} = \frac{S_{\text{i}}^{\alpha-\beta}}{1 - X_j^{\alpha-\beta}\left(1 - e^{-\frac{1}{X_j^{\alpha-\beta}}}\right)} \tag{3}$$

3 PROPOSED APPROACH

In this section, we present our ML-assisted approach for gate-level timing prediction. First, we describe our iterative algorithm that considers the strong interdependencies of Eq. (1, 2, 3) for initial timing estimation and feature extraction. Then, we assess different ML models and perform comparisons with features used in prior works.

3.1 Initial iterative refinement algorithm

The details of our initial iterative algorithm for post-PnR stage timing estimation are described in Algorithm 1. First, C_{eff_i} of each node i is initialized to C_{tot_i} for each voltage region, using the NLDM capacitance for receivers (step 0). In the first step of the main iterative loop, D_{drv} and S_{drv} are estimated by computing the driver output PWL voltage ramp using CSMs and multiple C_{eff_0} values (step 1). In step 2, during a forward breadth-first search (BFS) traversal of the RC-tree, the modified Elmore delay ($D_{0\rightarrow i}$) from driver output to node i is computed using Eq. (2), while the multiple slew values (S_i) of node i are propagated to each fanout node j using Eq. (3). When the multiple slew values of the PWL voltage ramps have been propagated to the target and side receivers, their C_{eff} values for all regions are updated using the CSM receiver model, which considers the Miller effect (step 3). Finally, in step 4, the RC-tree is traversed backward using BFS to update C_{eff_i} per region based on Eq. (1). This iterative refinement of C_{eff}, delay, and slew values is performed until the multiple driver output C_{eff_0} values converge within a specified tolerance (e.g., 0.01 fF), which is typically achieved in 5 iterations.

Algorithm 1: Stage timing estimation and feature extraction using CSM

Function `stage_CSM_timing`(*stage, multiple* $(\alpha V_{dd} - \beta V_{dd})$ *regions*):

0. Initialization:
- Set all $C_{\text{eff}_i}^{\alpha-\beta}$ values for each RC-tree node i to C_{tot_i} (using NLDM capacitance for receivers)

while *driver output* $C_{eff_0}^{\alpha-\beta}$ *values not_converged* **do**

1. Compute driver CSM PWL voltage ramp using $C_{\text{eff}_0}^{\alpha-\beta}$ values
2. Forward Traversal (driver-to-receivers):
- Compute Elmore delay ($D_{0\rightarrow i}$) to node i using Eq. (2)
- Compute $S_j^{\alpha-\beta}$ values for each fanout node j using Eq. (3)
3. Update $C_{\text{eff}_i}^{\alpha-\beta}$ values on receivers using CSM receiver model
4. Backward Traversal (receivers-to-driver):
- Compute $C_{\text{eff}_i}^{\alpha-\beta}$ values of node i using Eq. (1)

end

End Function

It is worth noting that several other moment-based metrics have been evaluated for delay (e.g., D2M [10], WED [11]) and slew (e.g., SS2M [10], LnS [12]) propagation, along with PERI [21] to extend metrics to saturated ramp inputs. However, our metrics employed in step 2 of our proposed Algorithm 1 lead to better accuracy results, while they do not require expensive computation of moments.

3.2 Machine learning enhancements

Although Algorithm 1 provides better accuracy and efficiency than using a π-model approximation, it may fail to achieve CSM-level accuracy (2-3% error against SPICE). The source of inaccuracy stems from the assumption of instantaneous circuit response [6, 7], which in certain stage configurations proved to be invalid due to the extensive resistive shielding effect [1]. Thus, to alleviate this problem, after the convergence of Algorithm 1, we employ four predictive ML models, using features extracted by Algorithm 1, to refine the delay and slew of the driver and the interconnect, respectively. Note that Algorithm 1 was configured with three voltage regions, using a set of four thresholds $V = \{0, V_{\text{low}}, V_{\text{delay}}, V_{\text{high}}\}$ [2], since they were sufficient to achieve signoff accuracy after the ML correction.

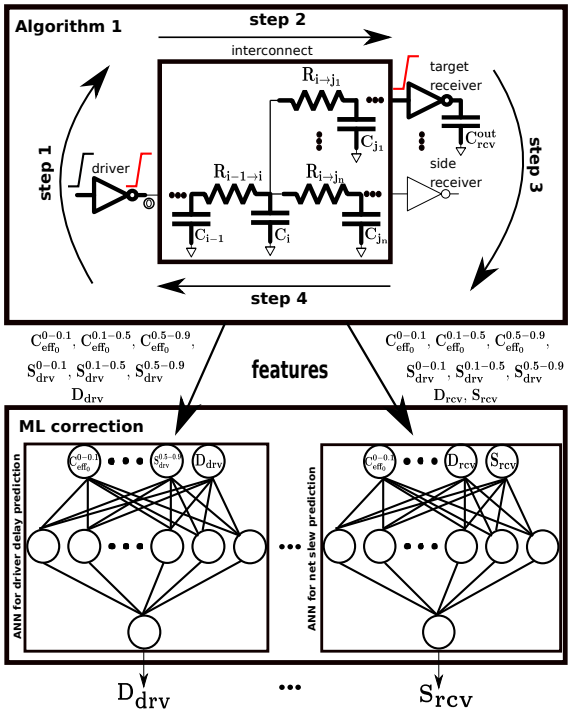

Figure 2: Proposed ML-enhanced approach.

3.2.1 ***Feature selection.*** The most crucial issue in building an ML model is finding the best features that describe the system of interest, not the ML model itself. The selection of features relied on our intuition that a suitable ML model could correlate the initial driver and interconnect delay/slew estimates of Algorithm 1 (i.e., D_{drv}, S_{drv}, D_{rcv}, S_{rcv}) with the golden SPICE values, given a representative description of the interconnect ($C_{\text{eff}_0}^{0-0.1}$, $C_{\text{eff}_0}^{0.1-0.5}$, $C_{\text{eff}_0}^{0.5-0.9}$), the initial driver output slew in regions ($S_{\text{drv}}^{0-0.1}$, $S_{\text{drv}}^{0.1-0.5}$, $S_{\text{drv}}^{0.5-0.9}$), the initial driver delay (D_{drv}) for driver delay/slew predictions, and the initial interconnect delay/slew (D_{rcv}, S_{rcv}) for interconnect delay/slew predictions. All the above features (shown in Figure 2) capture, in a compact way, the detailed *RC* values of the interconnect, the nonlinear behavior of signals, as well as the Miller effect. On top of that, our initial interconnect timing estimates also help to differentiate between receivers laying in different depths.

3.2.2 ***ML model selection.*** To assess the effectiveness of the chosen features in making predictions for the delay/slew of the

[2] In this work, we set $V = \{0, 0.1V_{\text{dd}}, 0.5V_{\text{dd}}, 0.9V_{\text{dd}}\}$, according to the examined library.

Table 1: Mean and maximum RE against SPICE (over the test set) along with memory requirements of the examined ML models

Features	Model	Mean RE against SPICE				Maximum RE against SPICE				Memory	
		D_{drv}	S_{drv}	D_{rcv}	S_{rcv}	D_{drv}	S_{drv}	D_{rcv}	S_{rcv}	D_{drv}/S_{drv}	D_{rcv}/S_{rcv}
Ours	ANN	1.1%	2.0%	1.2%	2.1%	86.3%	40.8%	98.5%	31.5%	3.62 KB	4.02 KB
	XGBoost	2.0%	2.7%	1.6%	2.6%	79%	39.7%	124%	32%	200.7 KB	193.7 KB
	LASSO	7.4%	7.6%	3.8%	6.1%	61.8%	61.2%	47.5%	41.9%	112 B	128 B
[14]	XGBoost	N/A	N/A	3.6%	2.4%	N/A	N/A	120%	32.1%	N/A	196.6 KB

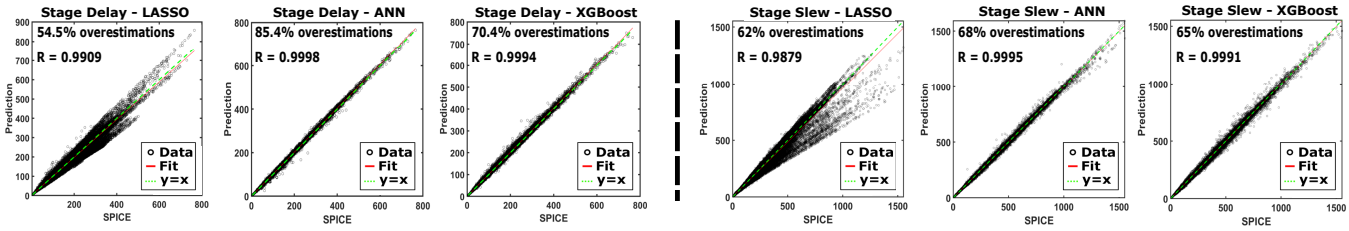

Figure 3: Stage delay and slew predictions (over the test set) of the proposed ML models using our features.

driver/interconnect, we investigated three different ML models, namely, the linear regression (LASSO), Artificial Neural Networks (ANNs), and XGBoost [16]. We evaluated these models on a dataset that includes rise/fall delay and slew values obtained by SPICE simulation of one million representative stages. [3] The examined models were trained on 80% and tested on 20% of the dataset. Before examining the effectiveness of the ML models, we applied a log-transformation to the features and the SPICE delay and slew values to transform their initial skewed distributions to normal distributions. All ML models were finely tuned through a grid-based optimization to obtain the best performance. The training time for the XGBoost model was 40 s, for the ANNs was 2 min, while it was negligible for the LASSO model. The XGBoost model was trained with 100 trees (estimators), while the ANNs were trained for 100 epochs. Any further increment in the number of trees or epochs did not improve accuracy but only exacerbated the training time. In addition, since overestimation (underestimation) of delay and slew is more desirable in a pessimistic setup (hold) STA, we also scaled our models' predictions by constant factors, without perturbing their accuracy. This approach was also the most effective in [15].

As listed in Table 1, ANNs outperform the other models in mean Relative Error (RE) and require much less memory than our XGBoost model. The LASSO model presents the lowest maximum RE in both D_{drv} and D_{rcv}, it is the most memory-efficient, but it is also the most inaccurate model. Furthermore, as shown in Figure 3, ANNs achieve the highest correlation with SPICE (over 0.9995) and lead to the most overestimations of stage delay (i.e., the sum of the driver and interconnect delay) and slew (i.e., the interconnect slew).

3.2.3 ***Assessment of features against prior works.*** To further evaluate our feature selection, we implemented an XGBoost model using the features of the state-of-the-art work [14] for interconnect timing estimation. Contrary to the 37 features used in [14], our models require only 8 features. More specifically, their features include i) the driver's drive strength and a single S_{drv} instead of our three ($S_{drv}^{0-0.1}$, $S_{drv}^{0.1-0.5}$, $S_{drv}^{0.5-0.9}$) values to characterize the driver waveform, ii) several moment-based metrics (e.g., Elmore, D2M) against our D_{rcv} and S_{rcv} as the target receiver's initial timing estimates, and iii) the resistance, downstream capacitance, and Elmore delay of each segment in the examined path instead of our three ($C_{eff_0}^{0-0.1}$, $C_{eff_0}^{0.1-0.5}$, $C_{eff_0}^{0.5-0.9}$) values to capture the interconnect complexity.

As shown in Table 1, our feature selection leads to more efficient and accurate ML models compared to [14]. The proposed XGBoost model results in 2% less mean RE in D_{rcv} predictions, almost the same mean RE in S_{rcv} predictions (2.6% over 2.4%), and similar maximum RE and model size. The benefits from our feature selection are more discernible in the ANN models. Our ANNs exhibit less mean RE by 2.4% and 0.3% for D_{rcv} and S_{rcv}, respectively, slightly better maximum RE, and require significantly (50×) less memory.

3.2.4 ***ANNs specifications.*** The architectures of all four ANNs (driver delay, driver slew, interconnect delay, interconnect slew), which achieve the best accuracy/runtime trade-off, consist of the input layer with the features, one hidden layer with 25 neurons, and the output layer with a single output. The fact that the best pick is a shallow ANN is in total agreement with the universal approximation theorem [22], which claims that every continuous function defined on a compact set can be arbitrarily well approximated by an ANN with one hidden layer. On top of that, the easy acquisition of statistical data and the small number of features (7-8) used to obtain compact sets in the feature space and reveal the continuity of delay/slew functions further strengthen the choice of a shallow ANN.

3.2.5 ***ML-enhanced stage timing prediction.*** In Figure 2, we provide a summary of our ML-enhanced approach. [4] As can be seen, we initially estimate the driver/interconnect delay/slew with Algorithm 1 and then refine these estimates using ANNs. It is worth mentioning that due to the statistical nature of ANNs (and any ML technique), the predictions for feature values located far away from the manifold implied by the training/test sets' points (that depend on the distribution of the random generator of stages) would be unreliable. So, in that case, we return the estimates of Algorithm 1 to guarantee robustness. Also, these values may be used to retrain the ANNs, and thus make more accurate predictions in the future.

[3] You can find details about the benchmarks and the experimental setup in Section 4.

[4] Our stage timing calculator is available at https://github.com/digaryfa/UTH-Timer.

4 EXPERIMENTAL EVALUATION

For the evaluation of the proposed ML-enhanced approach (ML), we compare it against our initial iterative method (i.e., Algorithm 1), a leading commercial STA tool in signoff mode (denoted as ST), and a modification of Algorithm 1 (denoted as "O'Brien") that approximates the input admittance of the interconnect by a π-model per region [17] and then computes C_{eff} using [7] in step 4 of each iteration. To compare these methods for both accuracy and efficiency, we integrated them into the C++ framework of the ACM TAU 2021 contest [20], which generates representative stages of VLSI circuits. For our experiments, the driver and receiver gates were selected from the ASU ASAP 7 nm FinFET Predictive PDK [18], while the golden results were obtained by Synopsys® HSPICE. The examined stages are split into five distinctive cases, listed in Table 2, each of 200k stages. The standard stages test typical signal propagation, whereas the stress cases are controlled topologies for more extreme conditions. All stages were generated so that driver output C_{eff}, driver input slew, and receiver output capacitance cover the entire range of the library's precharacterized LUTs and even beyond that. Finally, we ran all experiments on a Linux workstation with a 3.60 GHz 8-thread Intel® Xeon W-2123 CPU and 16 GB of memory.

Table 2: The testcases used for the experimentation

Testcase	Characteristics
standard	Multiple receivers randomly connected, while the target receiver is on the longest segment
stress_0	Point-to-point interconnect on a single layer for short distance timing propagation
stress_1	Point-to-point interconnect with multiple up-down layer transitions for long distance timing propagation
stress_2	Multiple receivers close to the driver for strong side load effect, followed by long distance timing propagation
stress_3	Long distance timing propagation, followed by multiple receivers for strong side load effect

4.1 Accuracy results

To evaluate the accuracy of the above methods, we measured their mean RE against SPICE for each timing metric (i.e., D_{drv}, S_{drv}, D_{rcv}, S_{rcv}), across all testcases cumulatively and for each testcase individually. As shown in Table 3, the O'Brien method fails to achieve CSM-level accuracy due to the significant error induced by the π-model approximation. Although Algorithm 1 leads to better accuracy than O'Brien, it is inadequate for signoff timing analysis, as its mean RE may be high (e.g., 7.11% for D_{drv}). Our proposed ML-assisted method achieves great correlation with SPICE, while also performing better than ST in D_{drv} (1.36% over 1.64%), D_{rcv} (1.30% over 1.62%), and S_{rcv} (2.54% over 2.76%) estimation. In addition, our method's mean absolute error against SPICE remains quite low, having 0.79 ps / 2.33 ps error for D_{drv} / S_{drv}, 1.30 ps / 2.59 ps error for D_{rcv} / S_{rcv}, and 0.90 ps error for stage delay.

In the second part of the accuracy evaluation, we compare our method against ST on a per-testcase basis, as shown in Table 4. More specifically, for the standard and stress_0 testcases, our methodology achieves similar accuracy with ST. As the testcases progressively become more complex (i.e., stress_1, stress_2, and stress_3), the estimates obtained by ML and ST start to diverge, with the best difference for ML against ST in stress_1 for S_{rcv} (2.11% over 3.96%) and the worst in stress_2 for S_{drv} (3.08% over 2.05%).

4.2 Runtime and memory results

The runtime and memory requirements of the examined methods are shown in Table 3. As can be seen, the O'Brien method is less efficient than Algorithm 1 and ML, as the π-model approximation per region requires additional computations during the backward traversal of the RC-tree, increasing runtime by 2× and memory by 1.8× compared to Algorithm 1. On the contrary, Algorithm 1 is the most efficient method, as it computes delay and slew for one million stages in 6.1 s while consuming 1.25 MB of memory. Finally, the addition of the ANNs to the proposed algorithm contributes a negligible overhead in memory (1.2%) and runtime (6%), rendering ML 35× faster than ST and six orders of magnitude faster than SPICE.

Table 3: Mean RE, memory requirements, and runtime of the examined timing estimation approaches across all testcases

Method	Mean RE agaist SPICE				Memory (MB)	Runtime (s)
	D_{drv}	S_{drv}	D_{rcv}	S_{rcv}		
O'Brien	9.48%	18.53%	5.90%	18.79%	2.25	12
Algorithm 1	7.11%	6.01%	5.52%	6.38%	1.25	6.1
ST	1.64%	1.95%	1.62%	2.76%	N/A	224.7
ML	**1.36%**	**2.38%**	**1.30%**	**2.54%**	**1.27**	**6.5**

Table 4: Mean RE of ML and ST for different testcases

Testcase	Method	Mean RE agaist SPICE			
		D_{drv}	S_{drv}	D_{rcv}	S_{rcv}
standard	ST	1.06%	1.25%	1.71%	1.34%
	ML	**1.17%**	**1.34%**	**1.68%**	**1.62%**
stress_0	ST	1.78%	1.81%	1.45%	2.42%
	ML	**1.17%**	**1.83%**	**1.15%**	**1.89%**
stress_1	ST	1.80%	2.45%	1.42%	3.96%
	ML	**1.90%**	**2.99%**	**1.15%**	**2.11%**
stress_2	ST	1.72%	2.05%	1.16%	2.72%
	ML	**1.24%**	**3.08%**	**1.12%**	**3.47%**
stress_3	ST	1.83%	2.17%	2.35%	3.37%
	ML	**1.34%**	**2.67%**	**1.40%**	**3.61%**

5 CONCLUSIONS

In this work, we presented a novel ML-enhanced approach for gate-level timing estimation using CSMs and the concept of multiple slew and C_{eff} values. Unlike previous works, we developed an iterative algorithm that extracts a few features that accurately represent the stage while also providing initial delay and slew estimates for both the driver and interconnect. Then, ANNs are employed to correlate the initial timing estimates with the golden SPICE values. Experimental results on stages implemented in 7 nm FinFET technology indicate that our method exhibits a strong correlation with SPICE (0.999), achieving 0.99% (0.90 ps) and 2.54% (2.59 ps) mean error for stage delay and slew, respectively. Moreover, the proposed approach is highly efficient (35× speedup over ST) and thus may be integrated into iterative timing-driven optimization steps to accelerate timing closure.

ACKNOWLEDGMENTS

This research has been co-financed by the European Regional Development Fund and Greek national funds via the Operational Program "Competitiveness, Entrepreneurship and Innovation", under the call "RESEARCH-CREATE-INNOVATE" (project code: T2EDK-00609).

REFERENCES

[1] J. Bhasker and R. Chadha, *Static Timing Analysis for Nanometer Designs: A Practical Approach*. Springer Science & Business Media, 2009.

[2] C. Bittlestone, A. Hill, V. Singhal, and A. NV, "Architecting ASIC libraries and flows in nanometer era," in *Proc. of the 40th Design Automation Conference (DAC)*, pp. 776–781, 2003.

[3] Cadence - Effective Current Source Model (ECSM). Accessed: Sep. 15, 2021. [Online]. Available: https://www.cadence.com/en_US/home/alliances/standards-and-languages/ecsm-library-format.html

[4] Synopsys - Composite Current Source (CCS). Accessed: Sep. 15, 2021. [Online]. Available: http://www.opensourceliberty.org/ccspaper/

[5] J. Qian, S. Pullela, and L. Pillage, "Modeling the "Effective Capacitance" for the RC Interconnect of CMOS Gates," *IEEE Trans. on Computer-Aided Design of Integrated Circuits and Systems (TCAD)*, vol. 13, no. 12, pp. 1526–1535, 1994.

[6] R. Puri, D. S. Kung, and A. D. Drumm, "Fast and accurate wire delay estimation for physical synthesis of large ASICs," in *Proc. of the 12th Great Lakes Symposium on VLSI (GLSVLSI)*, pp. 30–36, 2002.

[7] D. Garyfallou *et al.*, "Gate Delay Estimation With Library Compatible Current Source Models and Effective Capacitance," *IEEE Trans. on Very Large Scale Integration (VLSI) Systems*, vol. 29, no. 5, pp. 962–972, 2021.

[8] C. L. Ratzlaff and L. T. Pillage, "RICE: rapid interconnect circuit evaluation using AWE," *IEEE Trans. on Computer-Aided Design of Integrated Circuits and Systems (TCAD)*, vol. 13, no. 6, pp. 763–776, 1994.

[9] R. Gupta, B. Tutuianu, and L. T. Pileggi, "The Elmore delay as a bound for RC trees with generalized input signals," *IEEE Trans. on Computer-Aided Design of Integrated Circuits and Systems (TCAD)*, vol. 16, no. 1, pp. 95–104, 1997.

[10] K. Agarwal, D. Sylvester, and D. T. Blaauw, "Simple metrics for slew rate of RC circuits based on two circuit moments," in *Proc. of the 40th Design Automation Conference (DAC)*, pp. 950–953, 2003.

[11] F. Liu, C. V. Kashyap, and C. J. Alpert, "A delay metric for RC circuits based on the Weibull distribution," *IEEE Trans. on Computer-Aided Design of Integrated Circuits and Systems (TCAD)*, vol. 23, no. 3, pp. 443–447, 2004.

[12] C. Alpert, F. Liu, C. Kashyap, and A. Devgan, "Delay and slew metrics using the lognormal distribution," in *Proc. of the 40th Design Automation Conference (DAC)*, pp. 382–385, 2003.

[13] A. Kahng, S. Kang, H. Lee, S. Nath, and J. Wadhwani, "Learning-based approximation of interconnect delay and slew in signoff timing tools," in *Proc. of the International Workshop on System Level Interconnect Prediction (SLIP)*, pp. 1–8, 2013.

[14] H. Cheng, I. H. Jiang, and O. Ou, "Fast and Accurate Wire Timing Estimation on Tree and Non-Tree Net Structures," in *Proc. of the 57th Design Automation Conference (DAC)*, pp. 1–6, 2020.

[15] E. C. Barboza, N. Shukla, Y. Chen, and J. Hu, "Machine Learning-Based Pre-Routing Timing Prediction with Reduced Pessimism," in *Proc. of the 56th Design Automation Conference (DAC)*, pp. 1–6, 2019.

[16] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proc. of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794, 2016.

[17] P. R. O'Brien and T. L. Savarino, "Modeling the driving-point characteristic of resistive interconnect for accurate delay estimation," in *Proc. of the International Conference on Computer Aided Design (ICCAD)*, pp. 512–515, 1989.

[18] ASU - ASAP7 PDK. Accessed: Sep. 15, 2021. [Online]. Available: https://github.com/The-OpenROAD-Project/asap7/

[19] D. Garyfallou, I. Tsiokanos, N. Evmorfopoulos, G. Stamoulis, and G. Karakonstantis, "Accurate Estimation of Dynamic Timing Slacks using Event-Driven Simulation," in *Proc. of the 21st International Symposium on Quality Electronic Design (ISQED)*, pp. 225–230, 2020.

[20] TAU 2021 Timing Contest - Stage Delay Calculator using Current Source Models. Accessed: Sep. 15, 2021. [Online]. Available: https://sites.google.com/view/tau-contest-2021/

[21] C. Kashyap, C. Alpert, F. Liu, and A. Devgan, "Closed-form expressions for extending step delay and slew metrics to ramp inputs for RC trees," *IEEE Trans. on Computer-Aided Design of Integrated Circuits and Systems (TCAD)*, vol. 23, no. 4, pp. 24–31, 2004.

[22] K. Hornik, M. Stinchcombe, and H. White, "Multilayer feedforward networks are universal approximators," *Neural Networks*, vol. 2, no. 5, pp. 359–366, 1989.